MARKING SCHEME

ABERDARE TECHNICAL COLLEGE

Software cat 2

| Level 5, Module 3 | CAT 2, Term 3

Element 2: Test Computer Software Functionality | **Unit:** Install Computer Software (0619 451 06A)

Total Marks: 36 | **Time:** 1 Hour 30 Minutes

Question 1 (8 marks)

(a) Software testing (2 marks)

- Process of executing/evaluating a software application or system to find defects (1)
- To verify that it meets the specified requirements and works as expected (1)

(b) Definitions (3 marks, 1 each)

- **Error:** a human mistake made by a developer/analyst, e.g. wrong logic or misunderstanding a requirement (1)
- **Bug (defect):** a flaw in the code or design, caused by an error, that can make the software behave incorrectly (1)
- **Failure:** when the software does not perform as expected during execution, i.e. the visible result of a bug (1)

(c) HTTP methods in REST (3 marks, any three; 1 mark each = method ½ + purpose ½)

Method	Purpose
GET	Retrieve/read data from the server
POST	Create/send new data to the server
PUT	Update/replace an existing resource
PATCH	Partially update a resource
DELETE	Remove a resource

Question 2 (8 marks)

(a) Verification vs validation (4 marks)

- **Verification:** checking that the product is being built right, i.e. it conforms to the specification, without running the software (static: reviews, inspections, walkthroughs) (1)

- Example: reviewing the requirements document, design or code (1)
- **Validation:** checking that the right product is built, i.e. it meets the user's needs, by running the software (dynamic) (1)
- Example: user acceptance testing, system testing, or running the app and checking the output (1)

(b) Restaurant analogy for an API (4 marks)

- **Customer:** the client/application that makes a request (1)
- **Waiter:** the API, which takes the request to the system and brings the response back (1)
- **Kitchen:** the server/database/back-end that processes the request and prepares the data (1)
- **How it works:** request, then processing, then response; the client does not see the inner workings, only the menu (documented endpoints/rules) (1)

Question 3 (8 marks)

(a) Age field, 18 to 60 (4 marks)

(i) Equivalence partitioning (2 marks)

- Valid partition: 18 to 60 (1)
- Invalid partitions: below 18 (less than 18) and above 60 (greater than 60) (1)
- Accept non-numeric/blank as an extra invalid partition (no penalty if omitted)

(ii) Boundary value analysis (2 marks)

- Lower boundary: 17, 18, 19 (1)
- Upper boundary: 59, 60, 61 (1)
- Award ½ for any correct three of the six if the pairs are incomplete

(b) Postman requests, base URL https://school.com/api/ (4 marks; 1 each = method ½ + endpoint ½)

Task	Method	Suitable endpoint
(i) Retrieve list of students	GET	/students
(ii) Add a new student	POST	/students
(iii) Update student details	PUT (or PATCH)	/students/{id}, e.g. /students/5
(iv) Delete a student record	DELETE	/students/{id}, e.g. /students/5

Accept equivalent sensible endpoints (e.g. /student, /students/list).

Question 4 (8 marks)

(a) Black box vs white box (4 marks, 1 per heading: ½ black box + ½ white box)

Heading	Black box	White box
Knowledge of code	No knowledge of internal code or structure	Full knowledge of internal code and structure
Skills required	Testing and domain skills; no programming needed	Programming skills and understanding of the code
Coverage	Requirements/functional coverage (inputs and outputs)	Code coverage: statements, branches, paths, conditions
What it finds	Missing or incorrect functions, interface/UI errors, wrong outputs	Logic errors, dead/unreachable code, broken paths, security flaws in code

(b) Static vs dynamic testing, missing 12-character maximum check (4 marks)

- **Static testing (1½):** review/inspect the requirements, design or code without running it; the reviewer notices there is no condition limiting the length to 12 characters (e.g. no length <= 12)
- **Dynamic testing (1½):** run the form and enter a password longer than 12 characters (e.g. 13); the system accepts it, so the failure is observed
- **Why they complement each other (1):** static testing finds defects early and cheaply, before execution, but cannot show runtime behaviour; dynamic testing confirms actual behaviour and finds defects that reviews miss; together they give wider and more reliable defect detection

Question 5 (4 marks)

Bank system: SOAP (2 marks, any two reasons at 1 mark each; award ½ for choice if no reasons)

- Built-in security (WS-Security) for encryption, authentication and message integrity
- Supports ACID-compliant, reliable transactions (WS-AtomicTransaction/WS-ReliableMessaging)
- Strict contract (WSDL) and standards ensure consistency and formality
- Built-in error handling and auditing suited to financial data

Public mobile app backend: REST (2 marks, any two reasons at 1 mark each; award ½ for choice if no reasons)

- Lightweight and fast (typically JSON), using less bandwidth, which suits mobile devices
- Simple and easy to build and consume using standard HTTP methods
- Supports caching and scales easily
- Flexible data formats and no strict contract needed for simple data retrieval

Mark Summary

Question	Marks
1	8
2	8
3	8
4	8
5	4
Total	**36**

Award marks for correct alternative answers expressed in the candidate's own words.